

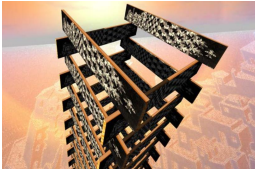
Application : analyse de logiciels embarqués avioniques critiques

TAS : Typage et analyse statique
M2, Master STL, Sorbonne Université

Antoine Miné

Année 2025–2026

Cours 14
29 janvier 2026



L'analyseur **Astrée** de programmes C

- Introduction
- Conception d'un **analyseur spécialisé** à l'analyse de **logiciels embarqués synchrones**
- **Exemples d'abstractions** utilisées dans Astrée
- Extension aux logiciels embarqués **concurrents**

Introduction

Vérification des logiciels avioniques

Les logiciels avioniques critiques doivent être **certifiés** :

- obligation légale
- processus régulé par des **standards internationaux** (DO-178B, DO-178C)
- **plus de la moitié** des coûts de développement
- essentiellement basé sur des campagnes massives de **tests** et sur la **relecture de code** par des pairs (donc par des humains)

Tendance :

l'utilisation de **méthodes formelles** est officiellement reconnue (DO-178C, DO-333)

- au niveau binaire, pour remplacer le test
- au **niveau du source**, pour **remplacer la relecture de code**
- au **niveau du source**, pour **remplacer le test**, mais uniquement si la **correspondance** entre source et binaire est aussi certifiée
- pour vérifier la robustesse et l'absence d'erreur à l'exécution

⇒ **les méthodes formelles permettent d'améliorer l'efficacité et de réduire le coût de la certification !**

Utilisation des méthodes formelles chez Airbus

Preuve de programme : (méthodes déductives : Hoare, Dijkstra)

- propriétés **fonctionnelles** pour des **petits morceaux de code C séquentiel**
- remplace le test unitaire
- **pas entièrement automatisé** (contrats, invariants et variants à spécifier)
- outils **Caveat** et **Frama-C** (CEA)

Analyse statique sûre : (interprétation abstraite)

- analyse **entièrement** automatique de **grosses applications C**, pour des propriétés **non fonctionnelles**
- sur du binaire séquentiel, analyse de temps d'exécution maximal et utilisation maximale de pile : **aiT**, **StackAnalyzer** (AbsInt)
- sur du C, absence d'erreur à l'exécution (dépassements de capacité, erreurs arithmétiques, dépassements de tableaux, etc.) : **analyseur Astrée** (ENS, AbsInt)

Compilation certifiée : (assistant de preuve)

- compilateur C **CompCert**, certifié en **Coq** (INRIA, AbsInt)
- équivalence entre **source** et **binaire** (s'il n'y a pas de comportement indéfini)
- permet à l'**analyse au niveau source** de **certifier aussi le binaire**

Analyse statique sûre

Avantages :

- analyse directe du code source (pas d'un modèle séparé)
- automatique (facile à utiliser par le programmeur, peu d'interaction nécessaire)
- efficace
- approximée (pour contourner les problèmes d'indécidabilité et de performance)

Sûreté :

- basée sur la sémantique (spécification C, entiers machines, flottants, pointeurs, ...)
- couverture totale du contrôle et des données
- les propriétés démontrées par l'analyseur sont vraies sur **toutes** les exécutions
⇒ aucune erreur n'est oubliée : **pas de faux négatif**
- la sûreté est **imposée** par le standard DO-178!

Rappel : conception « classique » d'un interprète abstrait

- 1 Écrire les règles de la sémantique concrète
- 2 Choisir la classe des propriétés d'intérêt
- 3 Déterminer la classe des propriétés qui doivent effectivement être inférées
- 4 Définir un analyseur sur une sémantique abstraite calculable

Rappel : conception « classique » d'un interprète abstrait



1 Écrire les règles de la sémantique concrète

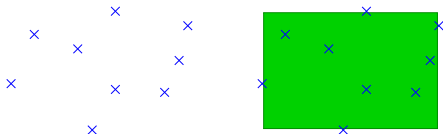
- fonction des programmes vers un monde mathématique riche
- formalisation fidèle de la spécification du langage
- vérité de base, sur laquelle la sûreté de l'analyse repose
- non calculable !

2 Choisir la classe des propriétés d'intérêt

3 Déterminer la classe des propriétés qui doivent effectivement être inférées

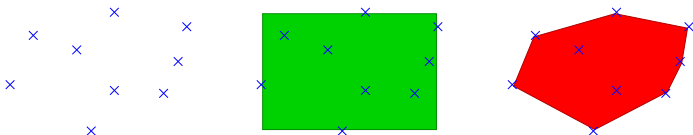
4 Définir un analyseur sur une sémantique abstraite calculable

Rappel : conception « classique » d'un interprète abstrait



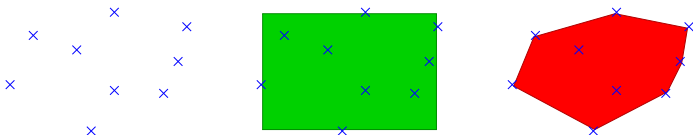
- 1 Écrire les règles de la sémantique concrète
- 2 Choisir la classe des propriétés d'intérêt
 - e.g. : bornes sur les variables, $X \in [a, b]$
- 3 Déterminer la classe des propriétés qui doivent effectivement être inférées
- 4 Définir un analyseur sur une sémantique abstraite calculable

Rappel : conception « classique » d'un interprète abstrait



- 1 Écrire les règles de la sémantique concrète
- 2 Choisir la classe des propriétés d'intérêt
- 3 Déterminer la classe des propriétés qui doivent effectivement être inférées
 - généralement, **plus expressive** que celle des propriétés d'intérêt
il faut représenter des invariants intermédiaires en tout point de programme,
des invariants de boucle, etc.
 - **peut dépendre** de la classe des programmes analysés
 - e.g. : contraintes linéaires, $\alpha X + \beta Y \leq \gamma$
- 4 Définir un analyseur sur une sémantique abstraite calculable

Rappel : conception « classique » d'un interprète abstrait



- 1 Écrire les règles de la sémantique concrète
- 2 Choisir la classe des propriétés d'intérêt
- 3 Déterminer la classe des propriétés qui doivent effectivement être inférées
- 4 Définir un analyseur sur une sémantique abstraite calculable
 - dériver ou inventer des opérateurs abstraits (e.g. : arithmétique d'intervalle)
 - inventer des opérateurs d'accélération ∇
 - domaine abstrait : structures de données et algorithmes

les calculs abstraits et ∇ accumulent des imprécisions
 $\Rightarrow$ nous ne trouverons pas la propriété la plus précise exprimable dans l'abstrait...

L'analyseur statique Astrée

L'analyseur statique Astrée

Analyseur statique de programmes temps-réels embarqués

- développé à l'**ENS** (2001–2009)
 - | B. Blanchet, P. Cousot, R. Cousot, J. Feret,
| L. Mauborgne, D. Monniaux, A. Miné, X. Rival
- industrialisé et commercialisé par **AbsInt**
(depuis 2009)



Astrée

www.astree.ens.fr



AbsInt

www.absint.com

Langage et propriétés

Sémantique concrète : définie par

- la **norme C99** (programmes portables)
- la **norme IEEE 754-1985** (calculs en virgule flottante)
- des paramètres spécifiques à chaque architecture
(sizeof, endianness, struct, etc.)
- des paramètres du compilateur et du linker (initialisation, etc.)

Propriétés d'intérêt :

- pas de **dépassement de capacité** en entier ni en flottant
- pas de **d'opération arithmétique invalide** (/0, << 33)
- pas de **d'accès de tableau ou de pointeur invalide**
- respect des assertions introduites dans le programme
par le programmeur (assert)

⇒ **absence d'erreur à l'exécution**

Interface

The screenshot displays the Astrée static analyzer interface. The main window is divided into several panes:

- Left Pane:** Contains a project tree for 'Example 1: scenarios' and a list of local settings (Preprocessing, Mapping to original sources, Reports, Analysis options, Parallelization, ABI, Global directives, General, Domains, Output). The 'Files' section shows 'scenarios.c' selected.
- Top Pane:** Displays the 'Analyzed file: /invalid/path/scenarios.c' and the 'Original source: C:/...ples/scenarios/src/scenarios.c'. The analyzed code shows a pointer 'ptr' and an array 'ArrayBlock' with some values highlighted in red. The original source code shows the same code with comments and a different value for 'ArrayBlock[15]'.
- Bottom Pane:** Shows a summary of errors and alarms. The 'Errors' section lists two errors: 'Overflow in conversion' and 'Possible overflow upon dereference'. The 'Alarms' section lists five alarms: 'Possible overflow upon dereference', 'Possible overflow upon dereference', 'Assertion failure', 'Define runtime error during assignment in this context. Analysis stopped for this context.', and 'Define runtime error during assignment in this context. Analysis stopped for this context.'.

At the bottom left, a status bar indicates 'Connected to localhost:1059 as anonymous@ABESINT-VMWARE'.

Applications d'Astrée



Airbus A340-300 (2003)



Airbus A380 (2004)



(modèle de) ESA ATV (2008)

- taille : de 70 000 à 860 000 lignes de C
- temps d'analyse : de 45mn à $\simeq$ 40h
- 0 alarme : preuve d'absence d'erreur à l'exécution

Analyseur statique **spécialisé** (1/2)

Principe :

- 1 Partir d'un analyseur **simple, rapide, peu précis** avec un domaine exprimant les propriétés d'intérêt (e.g., intervalles) et d'un programme caractéristique dans la classe d'intérêt

Analyseur statique **spécialisé** (1/2)

Principe :

- 1 Partir d'un analyseur **simple, rapide, peu précis**
avec un domaine exprimant les propriétés d'intérêt (e.g., intervalles)
et d'un programme caractéristique dans la classe d'intérêt
- 2 **Raffiner manuellement** l'analyseur jusqu'à atteindre 0 fausse alarme
 - déterminer quelles propriétés intermédiaires ne sont pas inférées
 - ajouter un nouveau domaine abstrait
 - si la propriété n'est pas déjà exprimable
 - utilisation d'opérateurs abstraits rapides et minimalistes,
 - limiter le périmètre d'activation du domaine (variables, portions de programme)
 - pour rester efficace
 - ou affiner des opérateurs abstraits dans les domaines existants
 - ou ajouter des réductions entre domaines existants
 - ou raffiner les paramètres de précision
 - périmètre d'activation des domaines, déroulements de boucles, degré partitionnement, élargissement, ...

Analyseur statique **spécialisé** (2/2)

Résultat

- analyseur **sûr** par construction
- **efficace** par parcimonie
- **0** fausse alarme sur le programme cible
- encourage une conception modulaire et des abstractions réutilisables

Justification théorique :

- pour chaque programme et propriété, un domaine adéquat existe
mais sa construction effective n'est pas mécanisable
- un domaine donné fonctionne sur un nombre infini de programmes
- toute combinaison finie de domaines échoue
sur un nombre infini de programmes

En pratique, **un analyseur sera précis sur toute une classe de programmes**

La suppression des fausses alarmes au cas par cas nécessite un affinage des paramètres de précision (qui peut souvent être effectué par l'utilisateur)

Spécialisation pour le code avionique séquentiel

Astrée est spécialisé pour :

- l'analyse des **erreurs à l'exécution**
dépassements arithmétiques, dépassements de tableaux, divisions par 0, etc.
- les logiciels **C embarqués critiques**
pas d'allocation dynamique de mémoire, pas de récursivité
- et en particulier les logiciels de **contrôle/commande**
programmes réactifs, avec des calculs flottants intensifs
structure de contrôle simple, structures de données simples
- l'analyse sûre et précise en vue d'une **validation**
toutes les erreurs sont trouvées, et peu de fausses alarmes

Environ **40 domaines abstraits** sont utilisés **simultanément**

Cible : codes réactifs synchrones conçus dans un langage graphique (e.g., Simulink) puis compilés en C

Exemple : diagramme de bloc d'un filtre digital

- chaque bloc est une (petite) fonction C prédéfinie
- une boîte a des **variables d'état** rémanentes (variables globales)
- les entrées sont volatiles (senseurs), bornées par des contraintes physiques
 $\implies$ très grand nombre d'exécutions possibles

Structure du code C généré

C généré

```
initialisation des variables d'état (globales)
while ( clock ≤ 3 600 000 ) {
    lire les entrées des senseurs (volatiles)
    calculer les sorties et le nouvel état
    écrire les sorties dans les actuateurs
    attendre le prochain tick d'horloge (≈ 10ms)
}
```

Caractéristiques :

- l'**espace d'état global** est très grand (≈ 10K variables)
- les structures de données restent **simples** (scalaires, tableaux)
- nombreux **calculs numériques flottants** (interpolation, filtrage digital)
- la structure de contrôle est **très plate**
(l'inlining des appels de fonctions est possible)
- **une très grande boucle** (≈ contenant tout le code)
avec des **boucles imbriquées petites et bornées** (déroulements possibles)

Note : sémantique **après** une erreur

Plusieurs sémantiques sont envisageables **après une erreur** :

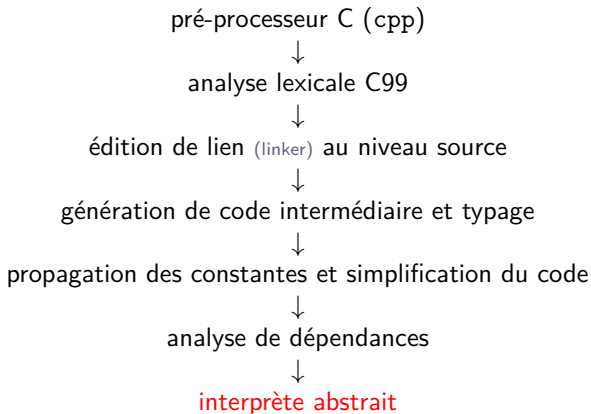
- **arrêter** le programme (i.e., l'opérateur retourne $\perp$)
 - division ou modulo par zéro
 - dépassement de capacité en flottant (selon la configuration du FPU)
 - échec d'un assert
- **retourner une valeur arbitraire**
(non déterminisme dans l'ensemble des valeurs autorisées par le type)
 - décalage de bit invalide
- **résultat bien défini**
 - arithmétique modulo 2^n en entiers non signés
 - *type-punning*
(ré-interprétation d'un motif de bits comme une valeur d'un autre type)
- **comportement totalement indéfini**
 - accès mémoire invalide

⇒ traité comme un arrêt de programme

Certains comportements peuvent être paramétrés par l'utilisateur

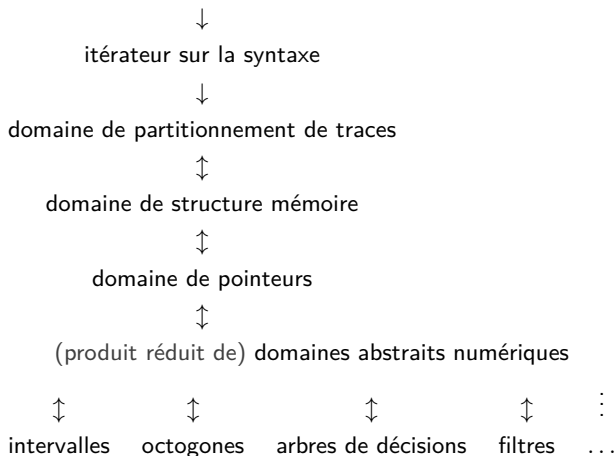
Toujours continuer l'analyse après une alarme si cela a un sens !

Vue générale de l'analyseur



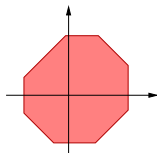
nombreux points communs avec le *front-end* d'un compilateur

L'interprète abstrait d'Astrée



Quelques abstractions utilisées dans Astrée

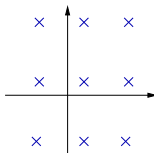
Quelques abstractions utilisées dans Astrée



octogones

$$\pm X \pm Y \leq c$$

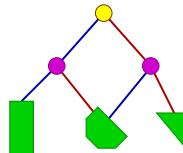
[Miné 2006]



congruences

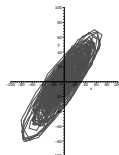
$$X \equiv a[b]$$

[Granger 1989]



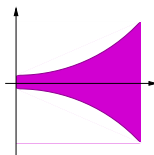
arbres de décision booléens

disjonctions
[Mauborgne]



ellipsoïdes

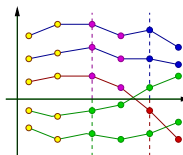
filtres digitaux
[Feret 2005]



exponentielles

$$X \leq (1 + \alpha)^{\beta t}$$

[Feret 2005]



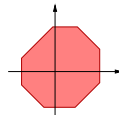
partitionnement de traces

disjonctions
[Mauborgne Rival 2005]

des domaines **relationnels** sont souvent nécessaires pour trouver des invariants induitifs adéquats

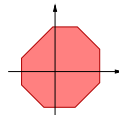
Paquets d'octogones

Invariants : $\bigwedge_{ij} \pm X_i \pm X_j \leq c_{ij}$: restriction des polyèdres
coût **cubique** (au lieu d'exponentiel), mais c'est **encore trop élevé** !



Paquets d'octogones

Invariants : $\bigwedge_{ij} \pm X_i \pm X_j \leq c_{ij}$: restriction des polyèdres
 coût **cubique** (au lieu d'exponentiel), mais c'est **encore trop élevé** !



Solution

Ne pas mettre toutes les variables $\mathbb{V}$ dans un **unique octogone**
 Créer à la place des octogones pour des **petits « paquets » de variables** :

- déterminés par une **pré-analyse de dépendance**
- ne relier dans un octogone que les variables modifiées ensemble
- interrompre les chaînes de dépendances aux frontières des blocs syntaxiques
 (évite de « tirer » toutes la variables dans chaque paquet)

Résultats : sur le type de programmes considérés

- nombre d'octogones linéaire en $|\mathbb{V}|$ et donc en $|P|$ (taille du programme)
- taille des octogones constante et petite ($\simeq 4$)

$\Rightarrow$ coût **linéaire en pratique**

Arbres de décision booléens : problème

Le flot de contrôle est souvent encodé dans des **variables booléennes**

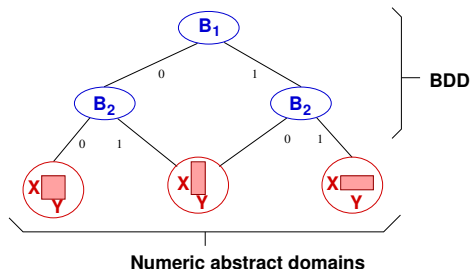
Exemple

```
B = (X > 0);
    ⋮ code long
    ⋮
if (B) Y = 1/X;
```

$(B = 1 \wedge X > 0) \vee (B = 0 \wedge X \leq 0)$ **non convexe**

Il est nécessaire de **partitionner** X par rapport à la **valeur de B**

Arbres de décision booléens : structure de données



BDD : structure **compacte** pour le partitionnement d'états

- variables **booléennes aux nœuds**
- **domaines numériques aux feuilles** (intervalles, octogones)
- **partage** des sous-arbres identiques $\Rightarrow$ meilleur efficacité

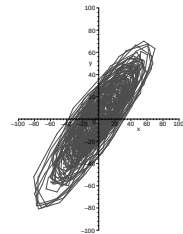
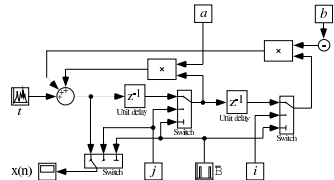
Mais il y a trop de variables booléennes pour un seul arbre :

- utiliser un **grand nombre d'arbres de petite taille** (profondeur 3)
- utiliser un critère syntaxique pour choisir les variables à relier

```
int INIT = 1;
float P, X, E1, E2, S1, S2;

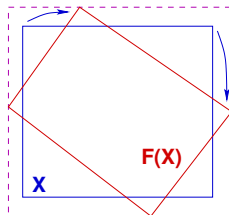
void filtre2() {
    if (INIT) {
        P = S1 = E1 = X;
    }
    else {
        P = (0.4677826 * X) -
            (E1 * 0.7700725) + (E2 * 0.4344376) +
            (S1 * 1.5419) - (S2 * 0.6740477);
    }
    E2 = E1;
    E1 = X;
    S2 = S1;
    S1 = P;
}

void main() {
    while (1) {
        X = input(); /* [-10,10] */
        filtre2();
        INIT = input(); /* [0,1] */
        wait_tick();
    }
}
```



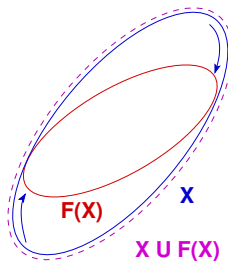
Filtrage numérique : domaine

- calcul de $X_n = \begin{cases} \alpha X_{n-1} + \beta X_{n-2} + Y_n \\ I_n \end{cases}$
- aucun **intervalle n'est stable**
- la surface stable la plus simple est une **ellipse** : $Y^2 - aYX - bX^2 \leq c$
un polyèdre stable pourrait exister, mais il aurait une forme compliquée ;
l'ellipse, elle, ne dépend que de 3 paramètres a , b et c , à inférer



$X \cup F(X)$

intervalle instable



$X \cup F(X)$

ellipse stable

Calculs flottants

Nombres flottants :

- cause d'**erreurs additionnelles** : dépassements de capacité
- **arrondi** : sûreté sur les réels $\nRightarrow$ **sûreté sur les flottants**
e.g. : `if (x != 0) y = 1 / (x*x)`

Calculs flottants

Nombres flottants :

- cause d'**erreurs additionnelles** : dépassements de capacité
- **arrondi** : sûreté sur les réels $\not\Rightarrow$ **sûreté sur les flottants**
e.g. : `if (x != 0) y = 1 / (x*x)`

Rendre les domaines sûrs sur les flottants :

- Intervalles
 - **facile** d'être sûr : **arrondi vers l'extérieur**
 $[a, b] \oplus [c, d] \stackrel{\text{def}}{=} [a \oplus_{-\infty} c, b \oplus_{+\infty} d]$

Calculs flottants

Nombres flottants :

- cause d'**erreurs additionnelles** : dépassements de capacité
- **arrondi** : sûreté sur les réels $\not\Rightarrow$ **sûreté sur les flottants**
e.g. : `if (x != 0) y = 1 / (x*x)`

Rendre les domaines sûrs sur les flottants :

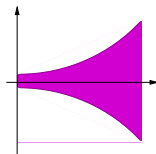
- Intervalles
 - **facile** d'être sûr : **arrondi vers l'extérieur**
 $[a, b] \oplus [c, d] \stackrel{\text{def}}{=} [a \oplus_{-\infty} c, b \oplus_{+\infty} d]$
- Domaines relationnels (octogones, polyèdres)
 - donner une sémantique **dans les réels** par transformation d'expressions en **ajoutant explicitement l'effet de l'arrondi** :
 $X \oplus Y \longrightarrow [1 - \epsilon, 1 + \epsilon]X + [1 - \epsilon, 1 + \epsilon]Y + [-\epsilon, \epsilon] \quad (\text{abstraction})$
 - passer ces expressions réelles à un **domaine sûr pour les réels**
 - le domaine lui-même peut être **implanté de manière sûre en flottants**
 $(X + Y \leq c) \wedge (-Y \leq d) \implies (X \leq c \oplus_{+\infty} d) \quad (\text{arrondi vers l'extérieur})$

Exponentielles

```

round.c
void main() {
  float X = input(); /* [0,100] */
  while (1) {
    X = X / 101.;
    ...
    X = X * 101.;
    wait_tick();
  }
}

```



Problème :

En flottant $(X/101) \times 101 \neq X$

$\times$ et $/$ ajoutent une **erreur d'arrondi**; X peut croître un peu à chaque itération
 $\Rightarrow$ la borne de X croît **exponentiellement** !

En pratique, la croissance est suffisamment **lente** et la durée d'exécution petite pour assurer qu'il n'y a **pas de dépassement de capacité**

Solution :

Domaine **relationnel non linéaire** pour relier X et tick

$$|X| \leq \alpha(1 + a)^{\text{tick}} + \beta \quad (\alpha, \beta \text{ et } a \text{ sont automatiquement inférés})$$

Réduction : $\text{tick} \leq \text{MAX_TICK} \Rightarrow$ borne sur X

Modèle mémoire bas niveau : problème

Type union

```
union {
    struct { uint8 al,ah,bl,bh } b;
    struct { uint16 ax,bx } w;
} r;
r.w.ax = 258;
if (r.b.al == 2) r.b.al++;
```

Type-punning

```
uint8 buf[4] = { 1,2,3,4 };
uint32 i = *((uint32*)buf);
```

Copie rapide

```
float a,b;
*((int*)&a) = *((int*)&b);
```

Norme C : programmes mal typés, comportements **indéfinis**

En pratique :

- il n'y a **pas d'erreur**
- la sémantique est **bien définie**

(spécification de l'*ABI*)

Modèle mémoire bas niveau : domaine

Type union

```
r.w.ax = 258;
if (r.b.al == 2) r.b.al++;
```

Principe :

- ne pas découper la mémoire en octets (peu précis pour les domaines numériques)
- mais découper en **cellules** représentant **un entier** (1, 2, 4 ou 8 octets)
- ajouter (et supprimer) **dynamiquement** les cellules à l'exécution en fonction des accès par pointeur
- une cellule $\simeq$ une « variable » pour les domaines numériques

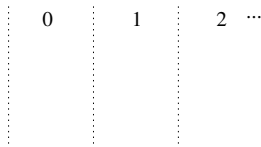
gestion automatique de la correspondance octet $\leftrightarrow$ cellule par le modèle sans intervention du domaine numérique :

- cellules qui se recouvrent $\implies$ conjonction de contraintes
- octets non couverts par une cellule $\implies \top$

Modèle mémoire bas niveau : domaine

Type union

```
r.w.ax = 258;
if (r.b.al == 2) r.b.al++;
```

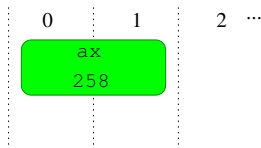


état initial : pas de cellule ($\top$)

Modèle mémoire bas niveau : domaine

Type union

```
r.w.ax = 258;
if (r.b.al == 2) r.b.al++;
```

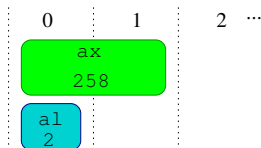


créer `r.w.ax` : cellule `uint16` à l'offset 0

Modèle mémoire bas niveau : domaine

Type union

```
r.w.ax = 258;
if (r.b.al == 2) r.b.al++;
```

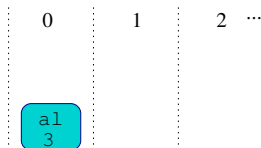


créer `r.b.al` : cellule `uint8` à l'offset 0
initialisée avec : `r.w.ax mod 256`

Modèle mémoire bas niveau : domaine

Type union

```
r.w.ax = 258;
if (r.b.a1 == 2) r.b.a1++;
```



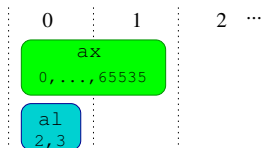
modifier la cellule `r.b.a1`

détruire la cellule invalide `r.w.ax`

Modèle mémoire bas niveau : domaine

Type union

```
r.w.ax = 258;
if (r.b.al == 2) r.b.al++;
```

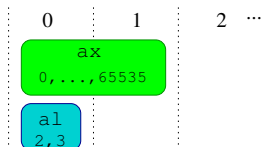


matérialiser les cellules dans toutes les branches
et faire l'union des environnements possibles

Modèle mémoire bas niveau : domaine

Type union

```
r.w.ax = 258;
if (r.b.al == 2) r.b.al++;
```



Dans l'abstrait : associer à chaque cellule

- un intervalle qui sur-approxime l'ensemble de ses valeurs possibles
- ou, plus généralement, une dimension dans un domaine numérique
e.g. polyèdres, pour garder des relations entre cellules

Extension à l'analyse de logiciels concurrents

Les logiciels concurrents

Programmation concurrente :

décomposer un programme en un ensemble de processus qui interagissent

- exploitation du parallélisme des ordinateurs (multi-cœurs, cloud)
- décomposition du programme en tâches asynchrones (serveurs, GUI, programmes réactifs, ...)

Dans l'avionique : *Integrated Modular Avionics*

- intégrer des fonctionnalités (moins de CPUs)
- remplacer les bus physiques par une mémoire partagée (moins de fils)
- pour les tâches moins critiques (DAL C-E, certification plus légère)
- allocation statique des ressources (threads, locks, mémoire)
- ordonnancement temps-réel (ARINC 653, POSIX threads real-time)

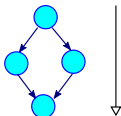
Difficultés :

- les logiciels concurrents sont plus difficiles à concevoir correctement
- et plus difficiles à valider et à vérifier
- le test est inefficace
- les méthodes formelles d'analyse de source sont peu développées

Retour sur l'analyse séquentielle

Deux méthodes d'analyse :

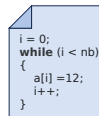
Équationnelle :



$$\begin{cases} X_1 = T \\ X_2 = F_2(X_1) \\ X_3 = F_3(X_1) \\ X_4 = X_2 \cup X_3 \end{cases}$$

- dériver un système d'équations du CFG
une variable $X_i \in \mathcal{D}^\sharp$ par point de programme i
- itérer (propager) le long du CFG
- coût mémoire linéaire en la **taille** du programme
- stratégie de propagation **flexible**
- facile à adapter aux programmes **concurrents**
avec un **produit** de CFG

Dénotationnelle :



$$\begin{aligned} C[\text{while } c \text{ do } b \text{ done}] X &\stackrel{\text{def}}{=} C[\neg c?] (\text{Ifp } \lambda Y. X \cup C[b?] (C[c] Y)) \\ C[\text{if } c \text{ then } t \text{ fi}] X &\stackrel{\text{def}}{=} C[t] (C[c?] X) \cup C[\neg c?] X \\ &\dots \end{aligned}$$

- méthode vue en cours et en TME
- itération sur la structure syntaxique
- coût mémoire linéaire en la **profondeur** du programme
- stratégie d'itération **fixée**
(suit la structure du programme)
- pas de définition inductive du parallélisme...

Méthode équationnelle séquentielle (exemple)

```

ℓ1 while true do
  ℓ2 if y < 100 then
    ℓ3 y ← y + rand(1, 3)

```

Système d'équations (concret) :

$$X_1 = \{[y \mapsto 0]\}$$

$$X_2 = X_1 \cup C[y \geq 100] X_2 \cup C[y \leftarrow y + \text{rand}(1, 3)] X_3$$

$$X_3 = C[y < 100] X_2$$

Le système est de la forme $\forall i \in \{1, 2, 3\}: X_i = F_i(X_1, X_2, X_3)$
et peut être résolu par itérations :

$$\begin{aligned}
 \forall i \in \{1, 2, 3\}: \quad & X_i^0 = \perp \\
 & X_i^{k+1} = F_i(X_1^k, X_2^k, X_3^k)
 \end{aligned}$$

Calcul identique dans le domaine abstrait, en utilisant un élargissement ∇ pour accélérer la convergence

Sémantique informelle des programmes *multi-thread*

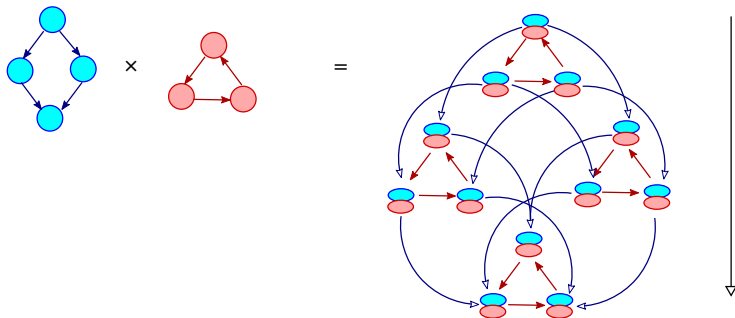
t_1	t_2
ℓ_{1a} while true do ℓ_{2a} if $x < y$ then ℓ_{3a} $x \leftarrow x + 1$	ℓ_{1b} while true do ℓ_{2b} if $y < 100$ then ℓ_{3b} $y \leftarrow y + \text{rand}(1, 3)$

Modèle d'exécution :

- ensemble fini de *threads* (logiciels embarqués)
- la mémoire est partagée (x, y)
- chaque thread a son propre compteur de programme
- l'exécution entrelace des pas de chaque thread t_1 and t_2 en choisissant **arbitrairement** la prochaine thread à exécuter (les affectations et les tests sont considérés comme atomiques)

$\implies$ nous avons l'invariant global : $0 \leq x \leq y \leq 102$

Analyse multi-thread par produit de CFG



Produit des CFG des threads du programme :

- état de contrôle = paire de points de programmes
 $\Rightarrow$ **explosion combinatoire** des états abstraits
- duplication des fonctions de transfert

$\Rightarrow$ exponentiel en le nombre de threads

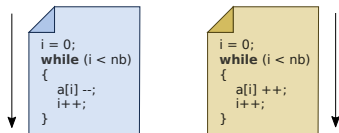
Analyse multi-thread par produit de CFG (exemple)

t_1	t_2
ℓ_{1a} while true do ℓ_{2a} if $x < y$ then ℓ_{3a} $x \leftarrow x + 1$	ℓ_{1b} while true do ℓ_{2b} if $y < 100$ then ℓ_{3b} $y \leftarrow y + \text{rand}(1, 3)$

Système d'équations (concret) :

$$\begin{aligned}
 X_{1a,1b} &= \{[x \mapsto 0, y \mapsto 0]\} \\
 X_{2a,1b} &= X_{1a,1b} \cup C[[x \geq y]] X_{2a,1b} \cup C[[x \leftarrow x + 1]] X_{3a,1b} \\
 X_{3a,1b} &= C[[x < y]] X_{2a,1b} \\
 X_{1a,2b} &= X_{1a,1b} \cup C[[y \geq 100]] X_{1a,2b} \cup C[[y \leftarrow y + \text{rand}(1, 3)]] X_{1a,3b} \\
 X_{2a,2b} &= X_{1a,2b} \cup C[[x \geq y]] X_{2a,2b} \cup C[[x \leftarrow x + 1]] X_{3a,2b} \cup \\
 &\quad X_{2a,1b} \cup C[[y \geq 100]] X_{2a,2b} \cup C[[y \leftarrow y + \text{rand}(1, 3)]] X_{2a,3b} \\
 X_{3a,2b} &= C[[x < y]] X_{2a,2b} \cup X_{3a,1b} \cup C[[y \geq 100]] X_{3a,2b} \cup C[[y \leftarrow y + \text{rand}(1, 3)]] X_{3a,3b} \\
 X_{1a,3b} &= C[[y < 100]] X_{1a,2b} \\
 X_{2a,3b} &= X_{1a,3b} \cup C[[x \geq y]] X_{2a,3b} \cup C[[x \leftarrow x + 1]] X_{3a,3b} \cup C[[y < 100]] X_{2a,2b} \\
 X_{3a,3b} &= C[[x < y]] X_{2a,3b} \cup C[[y < 100]] X_{3a,2b}
 \end{aligned}$$

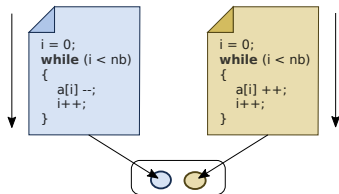
Analyse thread à thread avec interférences simples



Principe : éviter l'explosion combinatoire du contrôle

- analyser chaque thread **séparément**

Analyse thread à thread avec interférences simples

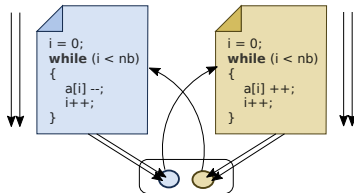


Principe : éviter l'explosion combinatoire du contrôle

- analyser chaque thread **séparément**
- **collecter** les **valeurs** écrites dans chaque variable par chaque thread
⇒ les **interférences**

abstraites dans un domaine abstrait, e.g., les intervalles

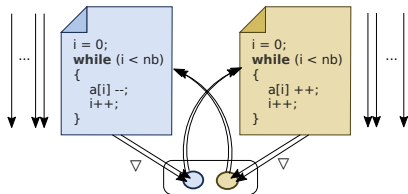
Analyse thread à thread avec interférences simples



Principe : éviter l'explosion combinatoire du contrôle

- analyser chaque thread **séparément**
- **collecter** les **valeurs** écrites dans chaque variable par chaque thread
 ⇒ les **interférences**
 abstraites dans un domaine abstrait, e.g., les intervalles
- **ré-analyser** les threads, en **injectant** ces valeurs à chaque lecture
 lire une variable retourne la dernière valeur lue,
 ou une interférence, de manière non déterministe
 ⇒ de nouveaux états et interférences sont découverts

Analyse thread à thread avec interférences simples



Principe : éviter l'explosion combinatoire du contrôle

- analyser chaque thread **séparément**
- **collecter** les **valeurs** écrites dans chaque variable par chaque thread
 ⇒ les **interférences**
 abstraites dans un domaine abstrait, e.g., les intervalles
- **ré-analyser** les threads, en **injectant** ces valeurs à chaque lecture
 lire une variable retourne la dernière valeur lue,
 ou une interférence, de manière non déterministe
 ⇒ de nouveaux états et interférences sont découverts
- **itérer** les analyses jusqu'à la stabilisation
 en appliquant un élargissement ∇ sur les interférences

Exemple d'analyse thread à thread

```

t1
ℓ1a while true do
  ℓ2a if x < y then
    ℓ3a x ← x + 1
  
```

```

t2
ℓ1b while true do
  ℓ2b if y < 100 then
    ℓ3b y ← y + rand(1, 3)
  
```

Analyse de t_1

(1a): $x = y = 0$

(2a): $x = y = 0$

(3a): $\perp$

Exemple d'analyse thread à thread

```

t1
ℓ1a while true do
  ℓ2a if x < y then
    ℓ3a x ← x + 1
  
```

```

t2
ℓ1b while true do
  ℓ2b if y < 100 then
    ℓ3b y ← y + rand(1, 3)
  
```

Analyse de t_2

(1b): $x = y = 0$

(2b): $x = 0, y \in [0, 102]$

(3b): $x = 0, y \in [0, 99]$

interférences découvertes : $y \leftarrow [1, 102]$

Exemple d'analyse thread à thread

```

t1
ℓ1a while true do
  ℓ2a if x < y then
    ℓ3a x ← x + 1
  
```

```

t2
ℓ1b while true do
  ℓ2b if y < 100 then
    ℓ3b y ← y + rand(1, 3)
  
```

Nouvelle analyse de t_1 avec interférences causées par t_2

interférences à appliquer : $y \leftarrow [1, 102]$

(1a): $x = y = 0$	
(2a): $x \in [0, 102], y = 0$	remplacer $x < y$ par $x < \max(y, [1, 102])$
(3a): $x \in [0, 102], y = 0$	remplacer $x \geq y$ par $x \geq \min(y, [1, 102])$

interférences découvertes : $x \leftarrow [1, 102]$

les analyses suivantes sont identiques : **le point fixe est atteint**

Exemple d'analyse thread à thread

```

t1
ℓ1a while true do
    ℓ2a if x < y then
        ℓ3a x ← x + 1
  
```

```

t2
ℓ1b while true do
    ℓ2b if y < 100 then
        ℓ3b y ← y + rand(1, 3)
  
```

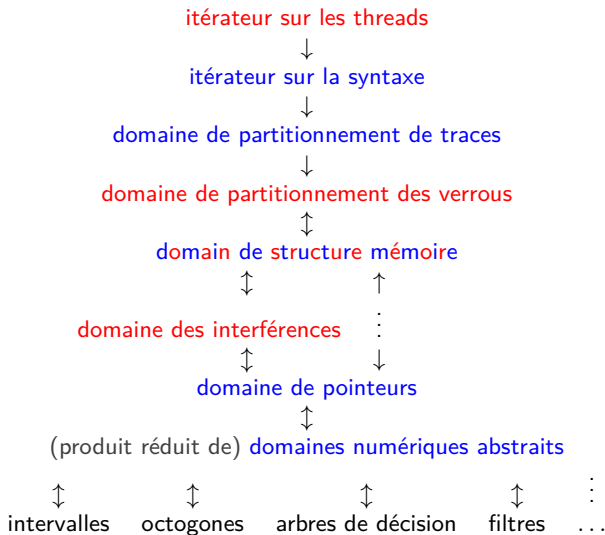
Invariants trouvés :

- nous avons $x, y \in [0, 102]$, mais pas $x \leq y$

Caractéristiques de l'analyse construite :

- similaire à une analyse de programmes séquentiels, mais itérée plusieurs fois paramétrée par un domaine abstrait arbitraire
- efficace (peu de ré-analyses nécessaires en pratique)
- les interférences sont non relationnelles et insensibles au flot de contrôle indépendamment du domaine numérique abstrait choisi, limite la précision

Interprète abstrait d'AstréeA



Famille cible d'applications

Modèle de concurrence :

- ensemble fixé de threads
- ordonnancement temps réel préemptif sur un seul processeur
- mémoire partagée, locks



Application cible :

- code avionique embarqué
- 1.6 Mloc de C, 15 threads
- code réactif + code réseau + listes, chaînes, pointeurs
- nombreuses variables, tableaux, boucles, graphe d'appel peu profond
- pas d'allocation dynamique de mémoire, pas de récursivité

Résultats

Efficacité :

- sur une machine Intel i7 2.90 GHz
- temps de calcul : 24h
- nombre d'itérations : 6 (pas d'élargissement nécessaire sur les interférences)
- 90 GB RAM

Précision : obtenue par spécialisation

- 12, 257 → 1, 195 fausses alarmes

L'amélioration de la précision est obtenue grâce :

- au partitionnement des interférences par les verrous
- à l'ajout d'interférences relationnelles
- à l'ajout de domaine abstraits spécialisés au domaine d'application

(non présentés ici)

Conclusion

Résumé

Objectif atteint

Astrée : il est possible de construire un analyseur statique à la fois :

- sûr vis à vis d'une sémantique réaliste du C
- raisonnablement efficace en temps et en mémoire
- précis sur une classe infinie de programmes
- utilisable dans un cadre de validation des logiciels critiques
- extensible à l'analyse efficace de logiciels concurrents

Recette

- partir d'un analyseur simple (intervalles)
- tant qu'il reste des fausses alarmes
 - chercher leur cause et, au choix
 - régler les paramètres d'analyse, ou
 - améliorer un domaine existant, ou
 - ajouter une réduction entre domaines existants, ou
 - ajouter un nouveau domaine